

Agent Smith

Autonomous reasoning, code generation, and execution

*Summary: In this project, you will build an **agentic framework** capable of autonomously solving coding challenges.*
Your agent will reason, write code, execute it in a sandboxed environment, and iterate until a solution is found.
*This project introduces **Code Agents**, **Model Context Protocol (MCP)**, and controlled code execution.*
Made in collaboration with @ldevelle

Version: 1.0

Contents

I Foreword **2**

II AI Instructions **3**

III Overview **5**
III.1 What is a Code Agent? 7

IV Common Instructions **9**
IV.1 General Rules 9
IV.2 Technical Constraints 9

V Mandatory Part **10**
V.1 Agentic Framework 10
V.1.1 Development Approach 12
V.2 The Sandbox 13
V.3 MBPP Agent 18
V.4 SWE-bench Agent 20
V.5 Mandatory Tools 25
V.5.1 File System Tools 25
V.5.2 Code Search Tools 25
V.5.3 Execution Tools 26
V.6 LLM API Providers 27
V.6.1 Examples of Free Providers 28
V.7 Model Benchmark Report 30

VI Evaluation **32**
VI.1 Hard Requirements and Limits 33
VI.1.1 MBPP Limits 33
VI.1.2 SWE-bench Limits 33
VI.2 Pass Criteria 34
VI.3 Grading Criteria 34
VI.4 What We Will Test in the Review 34
VI.4.1 AI Safety in Evaluation 35
VI.5 Evaluation logging structure 35

VII Readme Requirements **36**

VIII Submission **38**

Chapter I

Foreword

Software engineering is no longer only about writing correct code, it is about **understanding systems, navigating large codebases, debugging failures, and iterating efficiently**.

Modern Large Language Models have shown impressive capabilities in code generation, but code generation alone is not sufficient to solve real-world problems. Fixing bugs requires exploration, hypothesis testing, execution, observation, and refinement.

This is where **Agent Smith** come into play.
A Code Agent is not just a model that writes code: it is a system that can **reason about a task, interact with tools, execute code safely, observe results, and adapt its strategy**.

In this project, you will design and implement such a system. You will move beyond static prompts and JSON-based tool calling to build a **fully agentic loop** driven by executable Python code.

Welcome to the world of autonomous software agents.

Chapter II

AI Instructions

Context

AI is now a powerful coding partner — alongside your peers — for tackling large and demanding projects. You will guide it through both technical and non-technical aspects of your work.

AI tools can boost your efficiency and improve the quality of your output, but you should be able to dive deep into any part of the project without relying on them.

Your AI partner supports you, but you remain fully responsible for making informed technical decisions and to clearly explain and defend them.

Main message

- 👉 Strive for a mature and responsible use of AI.
- 👉 Never let AI take responsibility for decisions — especially when it lacks awareness of your goals, constraints, or team dynamics.
- 👉 Maintain creativity, innovation, and human oversight through active collaboration with your peers. AI is trained on existing data and rarely generates truly new ideas.
- 👉 Stay informed about emerging trends and be ready to adapt to new concepts and technologies.

Learner rules:

- Maintain intellectual leadership over your projects and make your own informed decisions.
- Prioritise the collective intelligence of your team and peers.
- Actively stay informed about the ongoing evolution of AI technologies.

Phase outcomes:

- AI engineering skills.
- Increased efficiency.
- Greater reliability and quality.
- A pioneering mindset.

Comments and examples:

- Your peers can identify trade-offs, question assumptions, and help you improve. The first answer from an AI might not be the best — it may lack efficiency, security, or real added value. Now more than ever, you should rely on your peers.
- AI can make you faster, but your peers make you better. Collaboration, discussion, and mutual challenge are key to success.
- Be transparent about how AI was used in your projects, and clearly identify what was generated by AI tools.

✓ Good practice:

> I asked AI to help generate unit tests for my API. I reviewed them with my teammate, and we adjusted them for edge cases. It saved time, and we both learned something new.

✗ Bad practice:

> I had AI generate the entire architecture of my project. It "works," but when I'm asked to explain design decisions during the peer review or in front from of a customer, I cannot. I lose credibility and I fail.

Chapter III

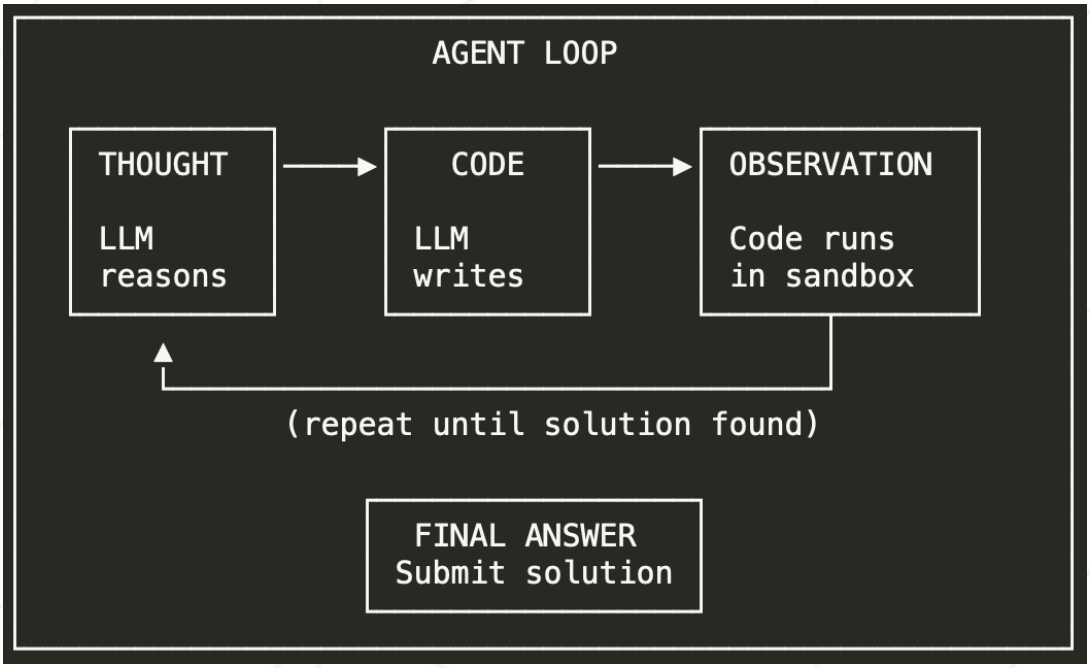
Overview

This project explores a new paradigm in AI-driven software engineering: **agentic code generation**.

Unlike classical approaches where an LLM outputs a final answer, your system will allow the model to:

- Write executable Python code
- Call tools directly from code
- Observe execution results
- Iterate until the task is solved

Your agent will operate through a structured loop: **Thought → Code → Observation**.



You will apply this framework to two different benchmarks:

- **MBPP**: algorithmic Python problems
- **SWE-bench**: real-world bug fixing in production repositories

The core challenge of this project is not only to make the agent intelligent, but also to make it **safe, controlled, reproducible, and measurable**.
In particular, you are expected to **evaluate and benchmark multiple language models** in order to analyze their performance and identify the most effective ones according to success rate and iteration efficiency.

III.1 What is a Code Agent?

A **Code Agent** is an AI system that can:

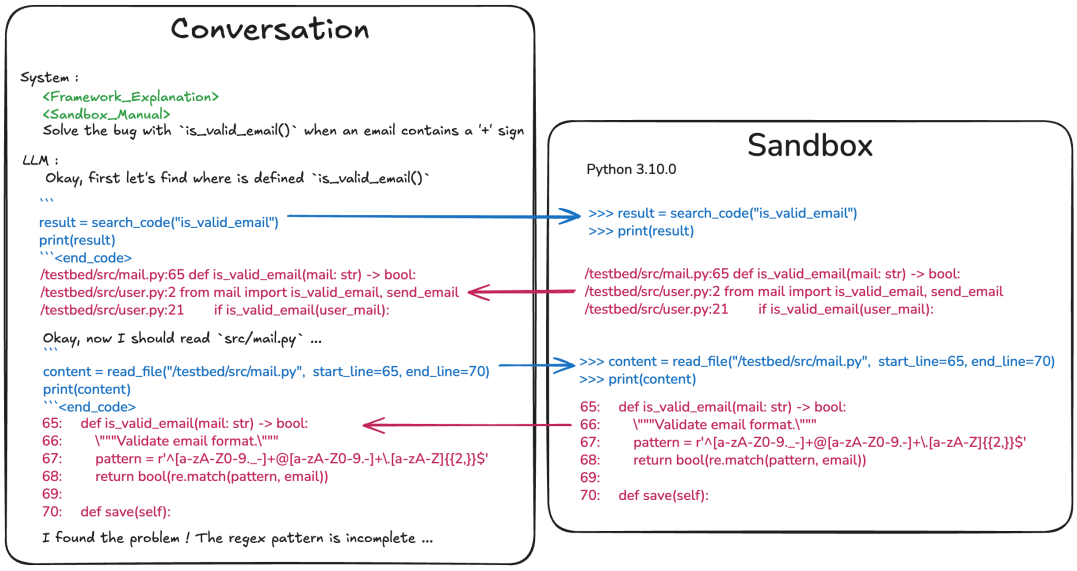
- Reason about a programming task
- Generate executable code
- Execute that code in a controlled environment
- Use tools to interact with files, tests, and repositories
- Observe the results and refine its approach

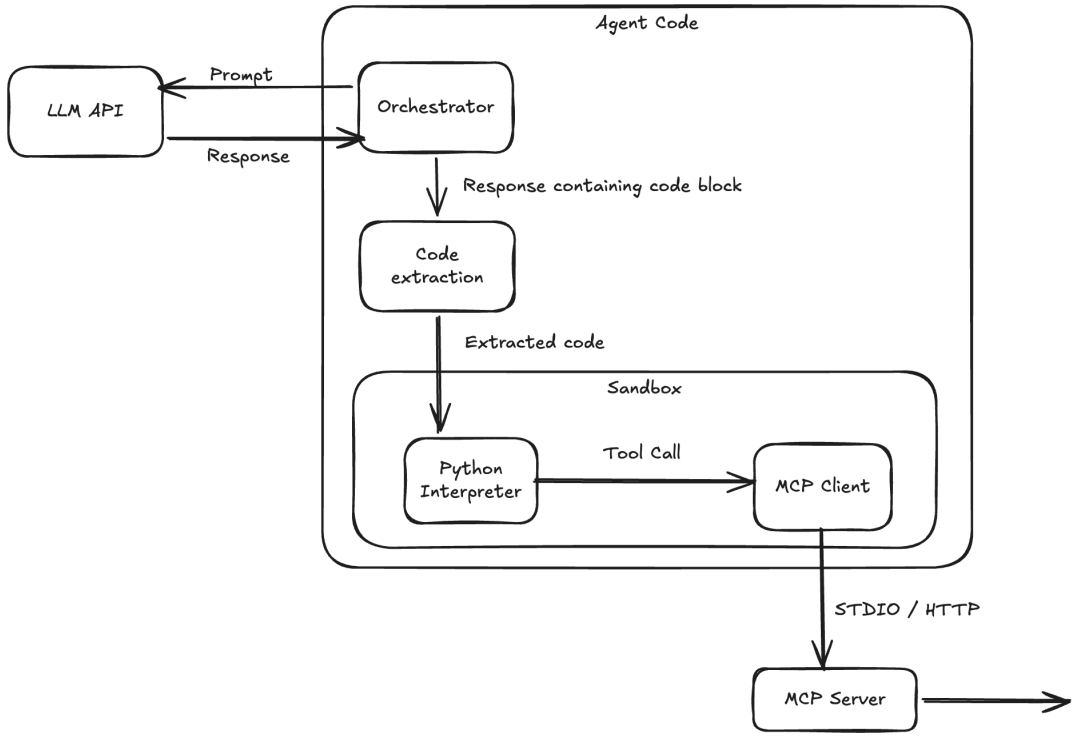
In this project, you will implement a **code-based tool calling** approach, where the LLM generates Python code such as:

```
result = search_code("validate_email")
print (result)
content = read_file("models.py", 1, 50)
print (content)
```

This approach is more expressive than JSON tool calling, allowing:

- Persistent variables between steps
- Conditional logic and loops
- Complex multi-step reasoning





1. **Agent/Orchestrator** is the central loop. It calls the LLM, extracts code, feeds it to the sandbox, reads observations, and repeats.

2. **Code Extraction** is a transform step between the LLM response and the sandbox.

3. **Sandbox** is the execution boundary. It enforces security restrictions on LLM-generated code. The sandbox contains an MCP client that connects to an external MCP server.

4. **final_answer()** is a built-in sandbox construct — it is NOT an MCP tool.

5. **MCP Server(s)** run as separate process(es) (via stdio or HTTP). Only your own MCP server(s) are allowed.

Chapter IV

Common Instructions

IV.1 General Rules

- You must use **Python 3.10**.
- You must use **uv** as your package manager.
- Your project must follow clean software architecture principles.
- All errors must be handled gracefully — crashes during evaluation will result in failure.
- Your code must be readable, structured, and documented.
- All execution must happen in a **sandboxed environment**.

IV.2 Technical Constraints

- You must support multiple LLM providers and models.
- You must implement usage tracking (tokens, retries, latency, requests).
- You must implement a configurable sandbox (imports, filesystem access), as specified in Section 4.2.
- Your tools must work independently of the agent loop.
- You must not use any library that re-implements agent orchestration logic (e.g., llama-index, smolagents, langgraph, crewai, autogen).
- The agent loop must be your own implementation.
- Even though the project can be completed without it, multi-agent architectures are allowed, but the orchestration must be your own code.

Chapter V

Mandatory Part

V.1 Agentic Framework

In this project, you must build an **agentic framework** capable of autonomously solving coding challenges.
For LLM provider selection, see Section 4.6.1 for examples of free-tier APIs.

Your system must:

1. Implement a **Thought → Code → Observation** loop
2. Extract LLM-generated Python code from the model responses



When benchmarking across multiple providers (Section 4.6.2), your extraction layer should handle different output formats that LLMs produce - as well as to fit what the LLMs you use were trained on - not just Python code blocks. A non-exhaustive list:

(a) **Python code blocks (primary)** - "‘python ... "‘ <end_code>

(b) **XML tool calls** - Anthropic-style <invoke name="..."><parameter>...</parameter></invoke>

(c) **JSON/Hermes tool calls** - <tool_call>{"name": "...", "arguments": {...}}</tool_call>

(d) **ReAct format** - Action: tool_name / Action Input: {...}

Non-Python formats should be converted to equivalent Python function calls (e.g., result = read_file(filepath="/testbed/file.py")) before sandbox execution. This lets the sandbox remain format-agnostic while supporting any model.



Think about it: What happens when your system prompt says "solve this bug" vs. when it demonstrates a step-by-step debugging workflow with a concrete example? What does the LLM need to know about available tools, expected output format, and problem-solving methodology? Compare the behavior of your agent with a vague prompt vs. an explicit one. The difference is often dramatic.

3. Execute the generated code inside a sandboxed environment
4. Feed the sandbox execution results back to the LLM
5. Solve benchmark tasks autonomously using the agent loop
6. Design the **system prompt**, including:
   - clear documentation of the available tools
   - examples of structured response slots (e.g. `Thought`, `Code`, `Observation`)
   - examples of effective agent reasoning loops



Your sandbox must provide explicit feedback to the LLM in all of these situations:

- No valid code block was found in the model's response
- A code block was malformed but was interpreted anyway (explain how)
- Execution hit the timeout and output is partial
- Tool output was truncated due to size limits
- An edit introduced a syntax error or lint violation

The LLM should never be left guessing about what happened. Silent failures lead to hallucinated observations and wasted iterations.

V.1.1 Development Approach

Building an autonomous agent is a complex task. Here are questions to guide your development:

Starting out:

- Which benchmark is simpler and should be tackled first?
- Am I testing with the most capable model available, or am I handicapping myself with constraints too early?
- Can my system solve the easiest task when I temporarily remove token and iteration limits? If not, adding constraints will only make it worse.

Debugging your agent:

- What happens in the first 3–5 iterations of my agent? Is it struggling with tool calling? Hallucinating observations? Following an unexpected path?
- Have I tried solving the target task myself, manually? What was my thinking process? Does my prompt reflect that methodology?
- How do I know if my agent is making progress on a task where tests still fail? Are there partial signals I could monitor?

Scaling up:

- My agent solves one task — does it generalize to others, or did I overfit?
- Am I optimizing too early (token usage, model choice), or have I proven the approach works first?
- Different models have different preferences for tool-calling format and orchestration style. Am I fighting the model's natural tendencies, or working with them?

V.2 The Sandbox

This part focuses on the design and implementation of a secure and configurable execution sandbox for the agent.

1. Implement a **sandbox CLI usage**

```
# Launch interactive sandbox
uv run sandbox

# With custom configuration
uv run sandbox sandbox_template.json

# With MBPP tools (stdio)
uv run sandbox --mcp-stdio "python mcp_tools_mbpp.py" sandbox_template.
    json

# With MBPP tools (HTTP)
uv run sandbox --mcp-server <URL>

# With SWE-bench tools
uv run sandbox --mcp-stdio "python mcp_tools_swebench.py"
    sandbox_template.json
```

2. Implement a `final_answer` tool
   What `final_answer` IS:
   `final_answer` is a callable function that the sandbox injects into the execution namespace. When the agent's code calls `final_answer(answer_string)`, the sandbox captures the argument and signals to the agent loop that the task is complete. The agent loop then terminates and produces the `SolutionOutput`.

   What `final_answer` is NOT: `final_answer` is **not** an MCP tool — it is not provided by any MCP server. It is always available in the sandbox namespace regardless of which MCP server is connected. MCP tools operate outside the sandbox (e.g., reading files in Docker, running tests), while `final_answer` operates within the sandbox to control the agent loop.

   How to use `final_answer`:

   - **MBPP**: call `final_answer(your_solution_code)`:
     pass your Python code as the argument
   - **SWE-bench**: call `final_answer(get_patch())`:
     pass the git patch retrieved via `get_patch()` as the argument

Architectural boundary:
The sandbox provides two kinds of callable functions in the execution namespace:

(a) **MCP tool wrappers**:
discovered dynamically from the connected MCP server

(b) `final_answer`: always present, provided by the sandbox itself

When you connect a different MCP server, the MCP tool wrappers change but `final_answer` remains.

Exception propagation: Your sandbox must correctly propagate exceptions that control program flow. In particular, `KeyboardInterrupt` and `SystemExit` must not be silently caught — they need to reach the agent loop for proper shutdown.

3. Enforce sandbox security constraints: Running LLM-generated code is inherently risky. Your sandbox is not just a technical component; it is a **safety boundary** between an autonomous system and the real world.

   - **Import restrictions**: Only modules from the configured allowlist may be imported
   - **Filesystem restrictions**: Access limited to allowed directories (e.g., `/testbed`, `/tmp/agent`)
   - **No network access**: Prevent any outbound or inbound network connections
   - **Execution timeout**: Terminate code exceeding the configured timeout (applies only to sandboxed code)
   - **Memory limits**: Terminate code exceeding allowed RAM usage
   - **Restricted builtins**: Remove or override dangerous builtins to prevent privilege escalation

4. Enforce sandbox limitations:

   - Restricted library imports
   - Restrict disk path acces
   - Execution timeout
   - Memory (RAM) limits

5. Integrate an **MCP server**
   - Mandatory tools must be integrated into the agent
   - MCP tools, resources, and prompts must be exposed
   - MCP tools must be callable as Python functions from the sandbox
   - The system will be tested with an **unknown MCP server**
   - The MCP tool files (mcp_tools_mbpp.py, mcp_tools_swebench.py) should be located at the root of your repository
   - Both stdio and streamable HTTP transports must be supported for MCP server connections
6. Generate a sandbox manual to be fed to the LLM prompt, which must include the MCP tools doc, or how to access it.



The sandbox manual should be dynamically generated from the connected MCP server's tool schemas - tool names, descriptions, and parameter types. When a different MCP server is connected, the manual should automatically reflect that server's tools.
The manual is what the LLM reads to understand what tools are available and how to call them.

Sandbox limits and behavior must be configurable using **Pydantic models and JSON configuration files**.

```
class SandboxConfig(BaseModel):
    """Sandbox configuration for student solutions.

    Uses allowlist approach: only imports in authorized_imports are allowed.
    Everything else is blocked by default.
    """
    authorized_imports: List[str] = Field(default_factory=lambda: [
        "math", "math.*",
        "collections", "collections.*",
        "itertools", "re", "json",
        "typing", "typing.*",
        "functools", "operator",
        "heapq", "bisect", "copy",
        "string", "random",
        "datetime", "datetime.*",
        "array", "cmath",
    ])
    allowed_directories: List[str] = Field(default_factory=lambda: [
        "/testbed", "/tmp/agent"
    ])
    max_execution_time_seconds: int = 30
```

```
    max_memory_mb: int = 512
```

When a different MCP server is connected, the sandbox should dynamically discover and expose that server's tools. Your mandatory tools are only present when your own MCP server is connected.



The sandbox is the central execution layer. It connects to an MCP server and exposes its tools as callable Python functions within the sandbox namespace. The sandbox wraps the MCP client, not the other way around. The sandbox and MCP tools are independent security domains: the sandbox restricts what LLM-generated Python code can do(imports, paths, timeout, memory), while MCP tool actions happen outside the sandbox.



The sandbox timeout only applies to code block executed within the sandbox. Actions performed by the MCP server (e.g., spawning external processes) are not subject to the sandbox timeout.



You are expected to implement the security mechanisms using only standard Python libraries and builtins. No external packages like RestrictedPython are allowed.

Sandbox isolation approaches



Think about it: Where should untrusted code execute; inside your process, or in a separate one? What does each choice mean for:

- Security boundaries (what can the code access?)
- Timeout handling (how do you kill runaway code?)
- Communication (how do you get results back?)

There are multiple valid approaches. Explore the trade-offs and choose what fits your architecture.

V.3 MBPP Agent

In this part, you will implement an autonomous agent dedicated to solving **MBPP (Mostly Basic Python Problems)** tasks.

1. Implement an **agent CLI interface**

```
# 1. Dump a task
cd moulinette
uv run moulinette_eval dump mbpp --output ../cache/mbpp_task.json

# 2. Run your agent
cd ../student
uv run python -m agent_mbpp --task-file ../cache/mbpp_task.json \
--output ../cache/mbpp_solution.json \
--model-name "model/name" --provider-url "https://provider.api/v1"

# 3. Validate solution
cd ../moulinette
uv run moulinette_eval validate mbpp ../cache/mbpp_task.json \
../cache/mbpp_solution.json
```

   - Task loading
   - Agent execution

2. Implement **MBPP MCP tools**
   - The `run_tests` tool
   - You may implement any additional tools you consider useful

3. Define **Pydantic models** for:
   - Task input

```
class MBPPTaskInput(BaseModel):
  """Input for MBPP task evaluation."""
  task_id: int
  task_definition: str
  function_definition: str
  test_imports: List[str] = Field(default_factory=list)
  test_list: List[str] = Field(default_factory=list)
```

- Agent output

```
class StepMetrics(BaseModel):
  """Metrics for a single agent step."""
  step: int
  input_tokens: int
  output_tokens: int
  request_time_ms: float
  api_url: str
  model_name: str
  llm_output: str
  sandbox_input: str
  sandbox_output: str
  retries: int
  timestamp: str = Field(default_factory=lambda: datetime.now().
     isoformat())

class SolutionOutput(BaseModel):
  """Output from student solution - this is what students must
     produce."""
  task_id: str
  benchmark: str # "mbpp" or "swebench"
  success: bool
  solution: str # Code for MBPP, patch for SWE-bench
  system_prompt: str
  iterations: int
  total_requests: int
  total_input_tokens: int
  total_output_tokens: int
  total_time_seconds: float
  steps: List["StepMetrics"] = Field(default_factory=list)
  error: Optional[str] = None
  timestamp: str = Field(default_factory=lambda: datetime.now().
     isoformat())
```

4. Design your agent to operate within the limits defined in Section 5.1 (iterations, tokens, time). The evaluation validates that your agent's output stays within these limits. `max_iterations` should be a configurable parameter of your agent loop.

External LLM providers are allowed, provided the interface remains compliant with the project constraints.

V.4 SWE-bench Agent

This part focuses on implementing an autonomous agent capable of solving **SWE-bench** tasks inside Dockerized environments.

For SWE-bench, both approaches are valid:
(a) deploy the sandbox inside the Docker container, or
(b) run the sandbox on the host with MCP tools bridging into Docker.

Regardless of choice, sandbox security constraints must be enforced.

- Fix real bugs or implement features in real repositories
- Explore codebases inside Docker containers, you are responsible to clean it after your program execution
- Generate and submit valid patches using 'git -c core.fileMode=false diff'



Mounting the testbed path (${TESTBED_PATH}) inside the container may be necessary for your tools to access the repository, depending on your implementation.



When iterating on SWE-bench, break the eval script into smaller goals. Track whether individual tests pass rather than waiting for the full script to succeed. Look at missteps in the first 1-5 iterations and improve your prompt/tools accordingly.

You may install additional dependencies inside the Docker container (e.g., ruff, jedi, tree) to enhance your tools.



What is the simplest agent that could solve the easiest task? What happens when you remove unused tools from your setup? For initial testing, try sympy__sympy-14711, sympy__sympy-13480, pydata__xarray-4629. What do existing open-source SWE agents look like when stripped to their essentials?

1. Implement an **agent CLI interface**

```
# 1. Dump a task
cd moulinette
uv run moulinette_eval dump swebench --output ../cache/swebench_task.json

# 2. Run your agent
cd ../student
uv run python -m agent_swebench --task-file ../cache/swebench_task.json \
--output ../cache/swebench_solution.json \
--model-name "model/name" --provider-url "https://provider.api/v1"

# 3. Validate solution
cd ../moulinette
uv run moulinette_eval validate swebench ../cache/swebench_task.json \
../cache/swebench_solution.json
```

2. Implement **SWE-bench MCP tools**
   - Tools must follow the specification defined in Mandatory Tools
   - You may implement any additional tools you consider useful
3. Define **Pydantic models** for:
   - Task input

```
class SWEBenchTaskInput(BaseModel):
    """Input for a SWE-bench task, provided by the moulinette.

    Your agent receives this and must produce a git patch that fixes
        the issue.
    """
    instance_id: str = Field(..., description="SWE-bench instance
        identifier (e.g., 'sympy__sympy-23534')")
    problem_statement: str = Field(..., description="The GitHub issue
         description, what needs to be fixed")
    docker_image: str = Field(..., description="Full Docker image
        name to pull (e.g., 'swebench/sweb.eval.x86_64.
        sympy_1776_sympy-23534:latest')")
    eval_script: str = Field(..., description="Bash script to run
        inside the container to evaluate the patch")
    hints_text: str = Field(default="", description="Optional hints
        about the issue (may be empty)")
    repo: str = Field(default="", description="Repository name (e.g.,
         'sympy/sympy')")
```

- Agent output (patch)

```
class StepMetrics(BaseModel):
    """Metrics for a single agent step.

    Each step corresponds to one LLM generate -> sandbox execute
        cycle.
    All fields are required for evaluation, empty strings are
        acceptable
    for steps where a field doesn't apply (e.g., no sandbox execution
        ).
    """
    step: int = Field(..., description="1-indexed iteration number")
    input_tokens: int = Field(..., description="Tokens sent to the
        LLM for this step")
    output_tokens: int = Field(..., description="Tokens generated by
        the LLM for this step")
    request_time_ms: float = Field(..., description="Wall-clock time
        for the LLM API call in milliseconds")
    timestamp: str = Field(default_factory=lambda: datetime.now().
        isoformat(), description="ISO 8601 timestamp of when this step
         was recorded")
    api_url: str = Field(default="", description="Base URL of the LLM
         API endpoint (e.g., 'https://openrouter.ai/api/v1')")
    model_name: str = Field(default="", description="Model identifier
         used for this step (e.g., 'qwen/qwen3-235b-a22b-2507')")
    llm_output: str = Field(default="", description="Raw text
        generated by the LLM before code extraction")
    sandbox_input: str = Field(default="", description="Python code
        sent to the sandbox for execution")
    sandbox_output: str = Field(default="", description="Sandbox
        execution result (stdout/stderr/error message)")
    retries: int = Field(default=0, description="Number of LLM API
        retries before getting a successful response (0 = first
        attempt succeeded)")


class SolutionOutput(BaseModel):
    """Output from student solution, required format for evaluation.

    This is the JSON structure your agent must produce and write to
        solution.json.
    The moulinette validates this against task correctness and
        metrics limits.
    """
    task_id: str = Field(..., description="Task identifier (MBPP
        task_id as string, or SWE-bench instance_id)")
    benchmark: str = Field(..., description="Benchmark type: 'mbpp'
        or 'swebench'")
    success: bool = Field(..., description="Whether the agent
        believes it solved the task")
```

```
    solution: str = Field(..., description="For MBPP: the Python
        function code. For SWE-bench: the git patch (diff)")
    iterations: int = Field(..., description="Number of agent loop
        iterations used")
    total_requests: int = Field(..., description="Total number of LLM
         API requests made (including retries)")
    total_input_tokens: int = Field(..., description="Sum of
        input_tokens across all steps")
    total_output_tokens: int = Field(..., description="Sum of
        output_tokens across all steps")
    total_time_seconds: float = Field(..., description="Wall-clock
        time from agent start to finish")
    steps: List[StepMetrics] = Field(default_factory=list,
        description="Per-step metrics, one entry per agent iteration")
    system_prompt: str = Field(default="", description="Full system
        prompt sent to the LLM (for provenance checking)")
    error: Optional[str] = Field(default=None, description="Error
        message if the agent failed (None if successful)")
    timestamp: str = Field(default_factory=lambda: datetime.now().
        isoformat(), description="ISO 8601 timestamp of when the
        solution was produced")
```

4. Enforce **hard limits** as defined in Section 5.1

External LLM providers are allowed, provided the agent remains fully compatible with the evaluation infrastructure.



Each step in the steps array of SolutionOutput must include api_url, model_name, fields identifying the LLM endpoint used for that step, plus llm_output (raw LLM response), sandbox_input (code sent to sandbox), sandbox_output (execution result), and retries (API retry count).



Think about it: Have you tried solving the target task manually, using only the tools your agent will have? What was your thinking process - which files did you read, what did you search for, how did you verify your fix? That process **IS the methodology** your prompt should teach the LLM.

`SolutionOutput` must include a `system_prompt` field containing the full system prompt sent to the LLM. Each step in `steps` must include `llm_output`, `sandbox_input`, and `sandbox_output` fields logging the raw LLM response, the Python code sent to the sandbox, and the execution result. A `retries` field tracks how many LLM API retries were needed for the step.

```
{
  "steps": [
    {
      "step": 1,
      "llm_output": "I'll read the file to understand the module.\n```python\
         nresult = read_file(filepath=\"/testbed/src/module.py\")\n```",
      "sandbox_input": "result = read_file(filepath=\"/testbed/src/module.py\")
         ",
      "sandbox_output": "def solve(x):\n return x + 1\n...",
      "retries": 0,
      "input_tokens": 1234,
      "output_tokens": 567,
      "..."
    }
  ]
}
```

This metadata is inspected during evaluation to verify the agent solved tasks through legitimate code exploration and reasoning, not by looking up solutions from pull requests, issues, or external sources.

V.5 Mandatory Tools

You must implement all the following tools. Each tool is exposed by the MCP (Model Context Protocol) server and will be tested independently in the context of the SWE-bench benchmark.

V.5.1 File System Tools

- `read_file(filepath, start_line, end_line)`
  Read the content of a file with line numbers.
  The output format must be similar to `cat -n`:

```
<line_number>: <line_content>
<line_number>: <line_content>
...
```

- `edit_file(filepath, old_str, new_str)`
  Replace an exact string in a file with a new string.

- `list_files(directory, pattern)`
  List files in a directory matching a given pattern.

V.5.2 Code Search Tools

- `search_code(pattern, file_pattern)`
  Perform a grep-like search in the codebase.
  The output must follow this format:

```
/absolute/path_to_file.py:<line_number> <line_content>
/absolute/path_to_other_file.py:<line_number> <line_content>
...
```

- `search_function_or_class_definition_in_code(name)`
  Find the definition of a function or a class.
  The output format must be similar to search_code format.

- `find_references(name, filepath, line)`
  Find all usages of a symbol (function or class).
  The output format must be similar to search_code format.

V.5.3 Execution Tools

- `run_tests()`
  Execute the evaluation script.

- `get_patch()`
  Retrieve the unified `git diff` of all changes made to the repository, depending on the implementation.

- `run_command(command, workdir)`
  Execute a shell command in the specified working directory.
  Returns the command's stdout, stderr, and exit code.

V.6 LLM API Providers

To implement your agent, you will need to perform calls to **Large Language Model (LLM) APIs**. Several providers offer free tiers or usage quotas that are sufficient for development and experimentation.

The list below is provided for **illustrative purposes only**. It is based on publicly available resources and may evolve over time.
You are free to use other providers as long as your system complies with the project requirements. (it must be free, must support multiple API tokens per provider)

The dataset your agent is evaluated on is called SWE-bench Verified. Knowing this, you can find:

- **Model leaderboards**: which LLMs perform best on real-world coding tasks?
- **Agent system descriptions**: how do the top-performing systems design their agent loop, tools, and prompts?
- **Per-task traces and evaluations** — how does a specific model perform on the exact task you're iterating on? This can help you choose the right model for your first debugging cycles.



What happens if the model keeps generating after a tool call, instead of waiting for real execution output? Most LLM APIs support a stop_sequences (or stop) parameter that tells the model to stop generating when it encounters a specific token sequence (e.g., <end_code>, </tool_call>, or a custom delimiter). Using stop sequences is essential to prevent the model from hallucinating tool outputs or continuing past the intended end of a code block. Without them, the model may generate fictional execution results instead of waiting for actual tool feedback.

V.6.1 Examples of Free Providers

- **OpenAI-compatible APIs**
  - OpenRouter — https://openrouter.ai
  - Together AI — https://www.together.ai
  - Groq — https://groq.com
- **Cloud providers with free access**
  - Google AI Studio (Gemini) — https://ai.google.dev
  - Mistral AI — https://mistral.ai
  - Cohere — https://cohere.com
- **Community or experimental APIs**
  - Fireworks AI — https://fireworks.ai
  - Perplexity AI — https://www.perplexity.ai
  - Anyscale — https://www.anyscale.com

Ensure your implementation supports **multiple API keys per provider** and consider implementing provider fallback. Rate limits and quotas may vary, especially under load during evaluation.

Important:

- This list is **not exhaustive and not contractual**.
- Access conditions, quotas, and available models may change over time.
- Learners are encouraged to explore additional providers or to use self-hosted models if needed.
- **The entire solution must rely exclusively on free tiers.**
  - No paid plans, purchased credits, or billing-enabled accounts are allowed.
  - The project must be fully executable using only free quotas at evaluation time.
- **Multi-token management is mandatory.**
  - Your system must support **multiple API tokens per provider**.
  - Token rotation must be implemented to handle rate limits and quota exhaustion.
- Your implementation must be sufficiently **abstract** to allow switching providers without major refactoring.



The choice of provider is not graded. The quality of your abstraction, error handling, and overall architecture is.

V.7 Model Benchmark Report



Think about it: How do you know which model is best for your agent? Raw solve rate tells part of the story - but what about exploration efficiency, token cost, provider reliability, and iteration discipline?

You must produce a benchmark report (`BENCHMARK_REPORT.md` at the root of your repository) comparing at least **5 models** on the same set of at least **3 SWE-bench tasks**. Your report must include:

1. **Setup**: Which models/providers were compared, which tasks were used, and why those tasks were selected
2. **Results table**: For each model × task combination:
   - Pass/Fail
   - Iterations used
   - Total input tokens
   - Total output tokens
   - Wall-clock time
3. **Provider reliability**: For each model/provider:
   - Average response time per request
   - Number of retries needed (rate limits, timeouts, errors)
   - Overall availability during benchmark runs
4. **Intermediary metrics** (at least 2 of the following):
   - Step at which the agent first reads/edits the file that appears in the final patch (exploration efficiency)
   - Step at which test failures first decrease vs baseline (partial progress)
   - Iterations between "tests first pass" and `final_answer` (submission discipline — zero is ideal)

   These metrics can be measured manually by inspecting your `solution.json` files — automation is not required. What matters is the analysis, not the tooling.
5. **Ablation study**: At least one before/after comparison of a change to your agent (prompt, tools, parameters) on the same tasks with the same model

6. **Conclusions**: Which model(s) you selected for your final pipeline and why. Which models can be disregarded and why. Based on your data.

The backing `solution.json` files must be present in your repository.

Chapter VI

Evaluation

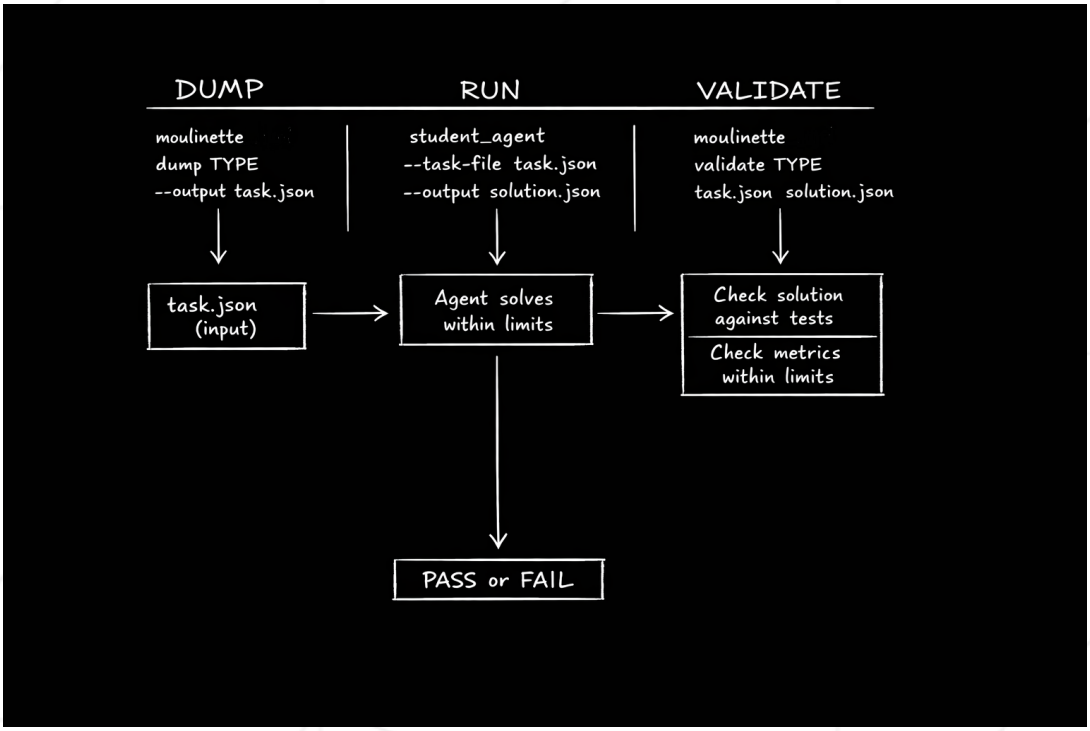
During evaluation, API keys and configuration are provided via a ‘.env‘ file passed as a required argument to evaluation scripts:

```
./exam_TYPE.sh --student-path ./student --moulinette-path ./moulinette --env-
   file /path/to/.env
```

Your CLI must support loading API keys from environment variables. Students should use standard environment variables (e.g., ‘OPENROUTER_API_KEY‘).

exam_mbpp.sh	**exam_swebench.sh**	**exam_sandbox.sh**
5 random tasks	3 random tasks	Security tests
Pass: 4/5	Pass: 2/3	Pass: ALL
For each task: 1. dump task 2. run agent 3. validate	For each task: 1. dump task 2. run agent 3. validate 4. container cleanup	Tests: - import block - builtin block - network block - path restrict - timeout - memory limit - MCP protocol
Limits: 10 iterations 6k input tokens 1.5k output tokens 120s timeout	Limits: 30 iterations 300k input tokens 10k output tokens 900s timeout	

Table VI.1: Summary of exam scripts, tasks, and sandbox/security limits.



VI.1 Hard Requirements and Limits

Your implementation must strictly respect the following execution limits. Exceeding any of these limits will result in a failure of the corresponding task.

VI.1.1 MBPP Limits

Metric	Limit
Maximum iterations	10
Maximum input tokens	6,000
Maximum output tokens	1,500
Timeout	120 seconds

VI.1.2 SWE-bench Limits

Metric	Limit
Maximum iterations	30
Maximum input tokens	300,000
Maximum output tokens	10,000
Timeout	900 seconds

- Token limits are **cumulative** across all iterations of a single task (all Thought → Code → Observation cycles combined).
- Thinking/reasoning tokens (when used by reasoning models) count toward the limit just like all other tokens.
- If your chosen model's reasoning tokens make limits tight, consider using a non-reasoning model.

VI.2 Pass Criteria

Benchmark	Tasks	Pass Threshold
MBPP	5 random tasks	4 out of 5
SWE-bench	3 random tasks	2 out of 3

No retries are allowed during examination.

VI.3 Grading Criteria

To pass the project, you must satisfy **all** of the following conditions:

- Meet the pass criteria for both MBPP and SWE-bench
- Respect all iteration, token, and timeout limits
- All mandatory tools pass independent tests
- The sandbox passes isolation and security tests



Hardcoded API keys in your code are a security failure. All API keys must be loaded from environment variables or configuration files (.env). Any API key found in your source code will be flagged during evaluation.

VI.4 What We Will Test in the Review

- Correctness of mandatory tools
- Correct implementation of the agent reasoning loop
- Sandbox security and isolation guarantees
- Model benchmarking results and token statistics
- Code quality, robustness, and overall architecture



During the evaluation, you will be asked to make small live modifications to your agent and re-run it on an MBPP task. This tests that you understand your own codebase and that the data in solution.json comes from real execution, not fabricated values. Each modification should take 2-5 minutes. If you cannot find where to make a change, it indicates you do not understand your own codebase. You will be asked to revert all changes after the exercise (e.g., git checkout).

VI.4.1 AI Safety in Evaluation

Building an autonomous code agent raises important safety considerations beyond just sandbox security. During evaluation, we verify that your agent solves tasks through legitimate reasoning and code exploration — not by exploiting shortcuts.

Your agent must **not**:

- Fetch solutions from pull requests, issues, or external sources
- Use memorized patches from its training data without genuine exploration
- Bypass the sandbox security constraints
- Access resources outside the provided task context

The `SolutionOutput` includes `system_prompt`, `llm_output`, `sandbox_input`, and `sandbox_output` fields precisely so that evaluators can trace the agent's reasoning process. This transparency is a core part of responsible AI development.

Violations will result in a grade of 0.

VI.5 Evaluation logging structure

Evaluation results are stored in:

```
./evaluations/EVAL_TYPE/YYYY-MM-DD_HH-MM-SS/task_id/task.json, solution.json,
   stdout.log, stderr.log
```



Your agent must solve tasks through reasoning and code exploration. Fetching solutions from pull requests, issues, or external sources is strictly forbidden and will result in a grade of 0.

Your CLI must support loading API keys from environment variables. Students should use standard environment variables (e.g., `OPENROUTER_API_KEY`).

Chapter VII

Readme Requirements

A `README.md` file must be provided at the root of your Git repository. Its purpose is to allow anyone unfamiliar with the project (peers, staff, recruiters, etc.) to quickly understand what the project is about, how to run it, and where to find more information on the topic.
The `README.md` must include at least:

- The very first line must be italicized and read: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].*
- A "**Description**" section that clearly presents the project, including its goal and a brief overview.
- An "**Instructions**" section containing any relevant information about compilation, installation, and/or execution.
- A "**Resources**" section listing classic references related to the topic (documentation, articles, tutorials, etc.), as well as a description of how AI was used — specifying for which tasks and which parts of the project.

➠ **Additional sections may be required depending on the project** (e.g., usage examples, feature list, technical choices, etc.).

Any required additions will be explicitly listed below.

Your `README.md` must include:

- System architecture
- Agent loop explanation
- Sandbox design
- Tool implementation details
- Benchmark results and analysis



```
Your README must be written in English.
```

Chapter VIII

Submission

Submit your project in your Git repository. Your repository must contain:

- The internal architecture and directory structure of your submission are left to you.
- Configuration files for sandbox and models
- `README.md`



`Do not include Docker images, large model weights, or generated outputs.`